

Cognitive Swarm Controller with Adaptive Rejoining Acceleration and Obstacle Braking

Complete Internal-to-Physical Update and ROS/Gazebo Interface

1 Purpose and relation to the fixed-speed model

The base cognitive-swarm model uses a physical state $(\mathbf{x}_i, \mathbf{e}_i)$ with $\|\mathbf{e}_i\| = 1$ and fixed translational speed,

$$\dot{\mathbf{x}}_i = v_0 \mathbf{e}_i. \quad (1)$$

The internal controller consists of directed Bloch-type perceptual channels, a slow regulatory state, cognitively weighted alignment, adaptive caution, and bounded turning. The present extension retains that architecture and adds a scalar physical speed $v_i(t)$. Acceleration is used selectively for two functions: (i) positive acceleration to rejoin a spatially split flock, and (ii) braking during sudden or strong avoidance manoeuvres. Ordinary coherent motion remains close to v_0 .

The complete agent state is therefore

$$\boxed{\mathcal{X}_i = \{\mathbf{x}_i, \mathbf{e}_i, v_i, s_i, \{\mathbf{m}_{ij}\}_{j \in \mathcal{N}_i}\}.} \quad (2)$$

2 Neighbour graph

For sensing radius R ,

$$A_{ij}(t) = \begin{cases} 1, & 0 < \|\mathbf{x}_i - \mathbf{x}_j\| < R, \\ 0, & \text{otherwise,} \end{cases} \quad \mathcal{N}_i(t) = \{j : A_{ij} = 1\}. \quad (3)$$

The graph is recomputed at every controller step.

3 Fast Bloch-type perceptual register

Each directed visible channel (i, j) carries

$$\mathbf{m}_{ij} = (m_x^{ij}, m_y^{ij}, m_z^{ij}). \quad (4)$$

The longitudinal effective field is

$$\boxed{h_z^{ij} = \kappa s_i + g_a \mathbf{e}_i \cdot \mathbf{e}_j + h_{\text{ext}}^{ij}.} \quad (5)$$

The fast state evolves as

$$\dot{m}_x^{ij} = -2h_z^{ij} m_y^{ij} - \frac{m_x^{ij}}{T_2}, \quad (6)$$

$$\dot{m}_y^{ij} = 2(h_z^{ij} m_x^{ij} - \Gamma m_z^{ij}) - \frac{m_y^{ij}}{T_2}, \quad (7)$$

$$\dot{m}_z^{ij} = 2\Gamma m_y^{ij} - \frac{m_z^{ij} - m_{z,\text{eq}}^{ij}}{T_1}, \quad (8)$$

with

$$m_{z,\text{eq}}^{ij} = \tanh h_z^{ij}. \quad (9)$$

The resolved alignment weight is

$$w_{ij} = A_{ij} \frac{1 + m_z^{ij}}{2}, \quad 0 \leq w_{ij} \leq 1. \quad (10)$$

Inactive channels make no alignment contribution. In the reference implementation their stored state is retained until the channel becomes visible again; resetting or passive relaxation can be substituted if desired for a particular sensor-memory interpretation.

4 Slow regulatory state and cognitive caution

The mean resolved alignment is

$$M_i = \begin{cases} \frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} m_z^{ij}, & |\mathcal{N}_i| > 0, \\ 0, & |\mathcal{N}_i| = 0. \end{cases} \quad (11)$$

The slow target and slow-state dynamics are

$$s_i^* = \tanh(\lambda_{\text{fb}} M_i), \quad (12)$$

$$\dot{s}_i = -\gamma_s(s_i - s_{\text{base}}) + \gamma_s s_i^*. \quad (13)$$

The state-dependent caution factor is

$$q_i(s_i) = 1 + c_q \sigma(s_i), \quad \sigma(s) = \frac{1}{1 + e^{-s}}. \quad (14)$$

Thus the closed loop is

$$\mathbf{m}_{ij} \rightarrow M_i \rightarrow s_i \rightarrow h_z^{ij}, q_i \rightarrow \mathbf{m}_{ij},$$

with s_i affecting both future perceptual resolution and avoidance gain.

5 Physical steering field

The cognitively weighted alignment vector is

$$\mathbf{D}_i^{\text{align}} = k_{\text{align}} \frac{\sum_{j \in \mathcal{N}_i} w_{ij} \mathbf{e}_j}{\sum_{j \in \mathcal{N}_i} w_{ij} + \varepsilon}. \quad (15)$$

Local cohesion is

$$\bar{\mathbf{x}}_i = \frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} \mathbf{x}_j, \quad \mathbf{A}_i^{\text{coh}} = k_{\text{coh}}(\bar{\mathbf{x}}_i - \mathbf{x}_i), \quad (16)$$

with $\bar{\mathbf{x}}_i = \mathbf{x}_i$ for an empty neighbourhood.

5.1 Predictive inter-agent safety

A finite geometric safety distance d_{safe} is enforced between flock agents. For neighbour j , define

$$\mathbf{r}_{ij} = \mathbf{x}_j - \mathbf{x}_i, \quad \mathbf{v}_{ij}^{\text{rel}} = v_j \mathbf{e}_j - v_i \mathbf{e}_i. \quad (17)$$

The time of closest approach over a short horizon T_a is

$$\tau_{ij}^* = \text{clip}_{[0, T_a]} \left(-\frac{\mathbf{r}_{ij} \cdot \mathbf{v}_{ij}^{\text{rel}}}{\|\mathbf{v}_{ij}^{\text{rel}}\|^2 + \varepsilon} \right), \quad (18)$$

and

$$\mathbf{r}_{ij}^* = \mathbf{r}_{ij} + \tau_{ij}^* \mathbf{v}_{ij}^{\text{rel}}. \quad (19)$$

The controller uses the smaller of $\|\mathbf{r}_{ij}\|$ and $\|\mathbf{r}_{ij}^*\|$ as the effective separation d_{ij} . If $d_{ij} < d_{\text{safe}}$, a smooth repulsive term is applied,

$$\mathbf{A}_i^{\text{sep}} = q_i \sum_{j \in \mathcal{N}_i} k_{\text{sep}} S\left(\frac{d_{\text{safe}} - d_{ij}}{d_{\text{safe}}}\right) \hat{\mathbf{n}}_{ij}, \quad (20)$$

where $\hat{\mathbf{n}}_{ij}$ points away from the more threatening current or predicted relative position and $S(z) = 3z^2 - 2z^3$ on $[0, 1]$. Thus agents preserve a safety gap not only when already close, but also when their relative motion predicts an imminent close pass.

5.2 Predictive moving-obstacle navigation

For moving obstacle a with centre $\mathbf{o}_a$, velocity $\mathbf{u}_a$, and conservative collision radius ρ_a , let

$$\mathbf{r}_{ia} = \mathbf{o}_a - \mathbf{x}_i, \quad \mathbf{v}_{ia}^{\text{rel}} = \mathbf{u}_a - v_i \mathbf{e}_i. \quad (21)$$

The predicted time of closest approach over horizon T_p is

$$\tau_{ia}^* = \text{clip}_{[0, T_p]} \left(-\frac{\mathbf{r}_{ia} \cdot \mathbf{v}_{ia}^{\text{rel}}}{\|\mathbf{v}_{ia}^{\text{rel}}\|^2 + \varepsilon} \right). \quad (22)$$

Then

$$\mathbf{r}_{ia}^* = \mathbf{r}_{ia} + \tau_{ia}^* \mathbf{v}_{ia}^{\text{rel}}, \quad d_{ia}^* = \|\mathbf{r}_{ia}^*\| - \rho_a. \quad (23)$$

The more threatening of the current and predicted clearances is denoted d_{ia} , with outward unit direction $\hat{\mathbf{n}}_{ia}$. To avoid the inefficient behaviour of pure radial repulsion, the obstacle response contains both a clearance-producing radial component and a tangential passing component,

$$\mathbf{A}_i^{\text{obs}} = q_i \sum_a S\left(\frac{\ell_{\text{obs}} - d_{ia}}{\ell_{\text{obs}}}\right) (1 + c_v \nu_{ia}) [k_r \hat{\mathbf{n}}_{ia} + k_t \hat{\mathbf{t}}_{ia}]. \quad (24)$$

Here ν_{ia} is the positive closing-speed contribution. The tangent $\hat{\mathbf{t}}_{ia}$ is chosen consistently for the flock: agents pass below an obstacle lying above the instantaneous flock centre and above one lying below it. This group-consistent side choice reduces unnecessary splitting and oscillation around an oncoming vehicle.

The demonstration represents the flock as small car-like agents moving in the fixed task direction $\hat{\mathbf{m}} = (1, 0)$; there is no corridor, lane potential, periodic boundary condition, or coordinate recycling. Agent coordinates evolve continuously in the global plane. Only the plotting camera translates with the flock for visualization. The forward task contribution is simply

$$\mathbf{A}_i^{\text{goal}} = k_{\text{goal}} \hat{\mathbf{m}}. \quad (25)$$

The complete steering vector is therefore

$$\mathbf{D}_i = \mathbf{D}_i^{\text{align}} + \mathbf{A}_i^{\text{coh}} + \mathbf{A}_i^{\text{sep}} + \mathbf{A}_i^{\text{obs}} + \mathbf{A}_i^{\text{goal}}. \quad (26)$$

The target direction is

$$\hat{\mathbf{d}}_i = \frac{\mathbf{D}_i}{\|\mathbf{D}_i\|}, \quad (27)$$

with $\hat{\mathbf{d}}_i = \mathbf{e}_i$ if the norm is numerically zero.

6 Bounded heading actuation

In 2-D let $\theta_i = \text{atan2}(e_{iy}, e_{ix})$ and $\theta_i^* = \text{atan2}(\hat{d}_{iy}, \hat{d}_{ix})$. Then

$$\Delta\theta_i = \text{wrap}(\theta_i^* - \theta_i), \quad (28)$$

and the commanded yaw rate is

$$\boxed{\dot{\psi}_i^{\text{cmd}} = \text{clip}\left(\frac{\Delta\theta_i}{\Delta t}, -\omega_{\max}, \omega_{\max}\right)}. \quad (29)$$

For a point-agent simulation,

$$\theta_i^{n+1} = \theta_i^n + \dot{\psi}_i^{\text{cmd}} \Delta t, \quad \mathbf{e}_i^{n+1} = (\cos \theta_i^{n+1}, \sin \theta_i^{n+1}). \quad (30)$$

In Gazebo this angle integration is performed by the vehicle/autopilot, not by the cognitive controller.

7 Selective variable-speed extension

7.1 Braking demand

Use the directional disagreement already produced by the steering field,

$$\boxed{\mu_i = 1 - \mathbf{e}_i \cdot \hat{\mathbf{d}}_i, \quad 0 \leq \mu_i \leq 2.} \quad (31)$$

A sudden obstacle rotates $\hat{\mathbf{d}}_i$ away from the current heading; therefore μ_i rises automatically and can trigger braking without a separate danger state.

7.2 Rejoining demand

Let $\mathbf{x}_i^r$ denote a rejoining reference available to agent i . In a centralized simulation it may be the global swarm centroid

$$\mathbf{x}_c = \frac{1}{N} \sum_j \mathbf{x}_j. \quad (32)$$

For real robots, $\mathbf{x}_i^r$ should be supplied by a swarm-state aggregator, a distributed/multi-hop centroid estimate, or a last-known group reference. This dependency is explicit because two physically disconnected components cannot infer each other's centroids from purely local sensing alone.

Define

$$\hat{\mathbf{c}}_i = \frac{\mathbf{x}_i^r - \mathbf{x}_i}{\|\mathbf{x}_i^r - \mathbf{x}_i\| + \varepsilon} \quad (33)$$

and

$$\boxed{\rho_i = \tanh\left(\frac{\|\mathbf{x}_i^r - \mathbf{x}_i\|}{L_r}\right) [\mathbf{e}_i \cdot \hat{\mathbf{c}}_i]_+}. \quad (34)$$

Hence a separated agent accelerates only after it has turned approximately toward the rejoining reference. This prevents a speed boost in the wrong direction.

7.3 Desired speed and acceleration

The desired speed is

$$\boxed{v_i^* = \text{clip}_{[v_{\min}, v_{\max}]} \left\{ v_0 \left[1 + \eta_r \rho_i - \frac{\eta_b}{2} \mu_i \right] \right\}}. \quad (35)$$

The physical acceleration command is

$$\boxed{a_i^{\text{cmd}} = \text{sat}_{[-a_{\text{br}}, a_{\text{acc}}]} \left(\frac{v_i^* - v_i}{\tau_v} \right)}. \quad (36)$$

The interpretation is intentionally asymmetric:

$$\begin{aligned}\rho_i > 0, \mu_i \approx 0 &\Rightarrow \text{catch-up acceleration,} \\ \mu_i > 0 &\Rightarrow \text{evasive braking,} \\ \rho_i \approx 0, \mu_i \approx 0 &\Rightarrow v_i^* \approx v_0.\end{aligned}$$

For the point-agent verifier,

$$v_i^{n+1} = \text{clip}_{[v_{\min}, v_{\max}]} \left(v_i^n + a_i^{\text{cmd}} \Delta t \right), \quad (37)$$

$$\mathbf{x}_i^{n+1} = \mathbf{x}_i^n + v_i^{n+1} \mathbf{e}_i^{n+1} \Delta t. \quad (38)$$

8 Exact fixed-speed limit

Set

$$\boxed{\eta_r = \eta_b = 0} \quad (39)$$

and initialize $v_i(0) = v_0$. Then $v_i^* = v_0$, $a_i^{\text{cmd}} = 0$, and

$$v_i(t) = v_0, \quad \dot{\mathbf{x}}_i = v_0 \mathbf{e}_i. \quad (40)$$

Thus the previous constant-speed model is recovered exactly while all internal cognitive dynamics and the heading update remain unchanged.

9 Complete update order

At each controller cycle $n \rightarrow n + 1$:

1. Read physical states $(\mathbf{x}_i, \mathbf{e}_i, v_i)$, neighbour states, and obstacle states $(\mathbf{o}_a, \mathbf{u}_a, \rho_a)$.
2. Recompute A_{ij} and $\mathcal{N}_i$.
3. Compute h_z^{ij} for all active directed channels.
4. Integrate $(m_x^{ij}, m_y^{ij}, m_z^{ij})$ and obtain w_{ij} .
5. Compute M_i , s_i^* , update s_i , and obtain q_i .
6. Build weighted alignment, cohesion, separation, obstacle and optional task steering; form $\mathbf{D}_i$ and $\hat{\mathbf{d}}_i$.
7. Compute yaw-rate command from the bounded heading error.
8. Compute rejoin demand ρ_i and manoeuvre demand μ_i .
9. Compute v_i^* and rate-limited a_i^{cmd} .
10. Export the physical commands to the simulator/robot interface.

The information flow is

$$\boxed{\text{sensors/peers} \rightarrow \mathbf{m}_{ij} \rightarrow w_{ij} \rightarrow M_i \rightarrow s_i \rightarrow q_i \rightarrow \hat{\mathbf{d}}_i \rightarrow (\dot{\psi}_i^{\text{cmd}}, v_i^*, a_i^{\text{cmd}})}.$$

10 ROS 2/Gazebo-ready interface

The implementation is deliberately reduced to two self-contained Python files. The ROS 2 file does not integrate an artificial point-mass plant: it reads the physical state from odometry, updates all internal cognitive variables, computes the swarm control law, and publishes only physical command references.

For agent i , the ROS 2 node subscribes to

$$\text{/agent.i/odom} \quad (\text{nav_msgs/Odometry}), \quad (41)$$

from which it obtains $\mathbf{x}_i$, heading $\mathbf{e}_i$, and measured speed v_i . Moving obstacles are supplied on

$$\text{/swarm/obstacles} \quad (\text{std_msgs/Float64MultiArray}) \quad (42)$$

as flattened rows

$$[x_a, y_a, u_{ax}, u_{ay}, R_a], \quad (43)$$

so that measured obstacle velocity can be used in the short-horizon prediction $\mathbf{o}_a^p = \mathbf{o}_a + T_p \mathbf{u}_a$.

At each control cycle the node executes the complete chain

$$\boxed{\text{odometry/obstacles} \rightarrow \mathbf{m}_{ij} \rightarrow w_{ij} \rightarrow M_i \rightarrow s_i \rightarrow q_i \rightarrow \hat{\mathbf{d}}_i \rightarrow (v_i^*, a_i^{\text{cmd}}, \dot{\psi}_i^{\text{cmd}}) \rightarrow \mathbf{v}_i^{\text{cmd}}}.$$

The world-frame planar velocity reference is

$$\mathbf{v}_i^{\text{cmd}} = v_i^{\text{cmd}} \hat{\mathbf{d}}_i, \quad (44)$$

with v_i^{cmd} obtained by one acceleration-limited step toward v_i^* . The bounded yaw-rate reference is computed from the desired/current heading error.

The node publishes

ROS 2 output	Content
/agent_i/cmd_vel	geometry_msgs/TwistStamped; world-frame $v_x^{\text{cmd}}, v_y^{\text{cmd}}$ and $\dot{\psi}_i^{\text{cmd}}$
/swarm/diagnostics	flattened per-agent rows $[v_i^*, a_i^{\text{cmd}}, \dot{\psi}_i^{\text{cmd}}, \mu_i, \rho_i, s_i, q_i, M_i, n_i]$

Thus the variables directly available for Gazebo/robot experiments are

$$\boxed{v_i^{\text{cmd}}, \dot{\psi}_i^{\text{cmd}}, v_i^*, a_i^{\text{cmd}}, \mu_i, \rho_i, s_i, q_i, M_i, n_i}.$$

The present `publish_commands()` function uses `TwistStamped`. If PX4, ArduPilot, or a particular Gazebo bridge expects body-frame velocity or a different message type, only this output function needs adaptation; the cognitive and swarm equations remain unchanged. Vehicle mass, inertia, thrust, attitude response, and motor dynamics remain the responsibility of Gazebo/autopilot.

11 Two-file software implementation

Only two Python files are supplied:

File	Responsibility
<code>swarm_demo.py</code>	self-contained real-time 2-D validation: full internal cognitive dynamics, steering, rejoining acceleration, evasive braking, bounded physical update, two initially split groups and exactly two constant-speed moving obstacles entering from opposite sides with small randomized crossing angles; the obstacles are visually distinct from the agents and no corridor is imposed
<code>ros2_swarm_node.py</code>	self-contained ROS 2/Gazebo controller: odometry and obstacle input, persistent $\mathbf{m}_{ij}$ and s_i , full cognitive-to-physical update, <code>cmd_vel</code> publication, and diagnostic publication for every controlled agent

No auxiliary local Python modules are required. Model parameters are held in an internal `Config` dataclass inside each self-contained file. This duplication is intentional: the real-time verifier can run directly with NumPy/Matplotlib, while the ROS 2 file can be copied into a ROS package without dependency on a custom Python module tree.

11.1 Running the real-time verifier

Run the standalone verifier from a terminal with

```
python swarm_demo.py
```

The program keeps an explicit live reference to the Matplotlib `FuncAnimation` object until the plotting window is closed. This is necessary because an anonymous animation object may otherwise be garbage collected before its first frame is rendered, producing Matplotlib’s “Animation was deleted without rendering anything” warning. The call to `plt.show(block=True)` keeps the GUI event loop active for the duration of the demonstration. A normal interactive Matplotlib backend (for example TkAgg, QtAgg, or the macOS backend) is therefore required for the live plot.

11.2 Unbounded longitudinal evolution and split–reformation diagnostic

No periodic boundary condition is used for the flock. The physical update is always

$$\mathbf{x}_i^{n+1} = \mathbf{x}_i^n + v_i^{n+1} \mathbf{e}_i^{n+1} \Delta t, \quad (45)$$

and the resulting global coordinates are retained for the full simulation. For visualization only, the plotting window is centred on the instantaneous flock centroid,

$$\mathbf{x}_c = \frac{1}{N} \sum_i \mathbf{x}_i, \quad (46)$$

so a continuously advancing flock remains visible without altering its state.

The two demonstration obstacles have constant speed magnitude u_0 but a slowly varying heading,

$$\mathbf{u}_a = u_0(\cos \phi_a, \sin \phi_a), \quad \dot{\phi}_a = \omega_a(t), \quad (47)$$

where $\omega_a(t)$ is piecewise-random and the relative heading is bounded around the oncoming direction,

$$|\text{wrap}(\phi_a - \pi)| \leq \phi_{\max}. \quad (48)$$

Thus the vehicles remain globally oncoming but can weave laterally and generate nontrivial close approaches. After an obstacle passes a fixed distance behind the current flock centroid it is placed again far ahead of the *current* global flock coordinate with a new lateral offset and heading; this is obstacle generation, not a periodic boundary condition on the swarm.

To show the desired split–reformation behaviour explicitly, the demo reports the number of connected flock components under the sensing graph

$$A_{ij} = \mathbf{1}(\|\mathbf{x}_i - \mathbf{x}_j\| < R). \quad (49)$$

A close obstacle can temporarily increase the number of connected components; successful rejoining is indicated when the count returns to one, accompanied by a reduction of the flock span and rejoining demand ρ_i .

12 Reference defaults and tuned two-obstacle road-like scenario

The internal cognitive parameters remain close to the representative scales used in the present model ($T_1 = 0.6$, $T_2 = 0.3$, $\Gamma = 1$, $\kappa = 1.2$, $\gamma_s = 0.03$, $\lambda_{fb} = 2$). The supplied demonstration is intentionally an unbounded road-like encounter rather than a corridor-like arena: a compact flock of small agents advances indefinitely in global $+x$. Two larger unruly vehicles are repeatedly spawned well ahead of the current flock, travel generally in the opposite direction with constant speed magnitude, and execute a bounded random heading weave. Their lateral spawn positions are biased toward the flock so that close encounters can split the group. They are respawned only after passing well behind the moving flock.

The controller-level defaults are

Parameter	Value	Parameter	Value
Δt	0.04	R	12
d_{safe}	2.4	T_a	0.75 s
ℓ_{obs}	8.5	T_p	1.0 s
k_{sep}	3.4	k_r, k_t	5.2, 4.6
k_{goal}	1.65	ω_{max}	1.7 rad s ⁻¹
v_0	3.0	$v_{\text{min}}, v_{\text{max}}$	0.9, 4.6
η_r, η_b	0.55, 1.0	L_r	10
τ_v	0.42 s	$a_{\text{acc}}, a_{\text{br}}$	2.2, 4.5

The two demonstration obstacles use a larger car-like footprint (approximately 4.2×2.0 simulation units), constant speed magnitude $u_0 = 4.0$, and a conservative circular control envelope $\rho_a = 2.25$. Their heading is allowed to wander within approximately ± 0.55 rad of the oncoming direction. Flock agents are intentionally drawn much smaller (approximately 0.95×0.50 units visually) so that obstacle vehicles are immediately distinguishable from the swarm.

These are controller-level working defaults, not universal optima. The tuning emphasizes four objectives simultaneously: (i) no obstacle contact, (ii) preservation of a finite inter-agent gap, (iii) early smooth lateral navigation rather than last-second radial escape, and (iv) rapid recovery of flock compactness after the encounter. The tangential obstacle component is deliberately comparable to the radial component because a purely radial field tends to brake and scatter the flock around a large oncoming vehicle. Final Gazebo values must be retuned against actual vehicle dimensions, steering limits, sensing delay, and the precise frame conventions used by the simulator or autopilot.

13 Practical recommendation for Gazebo transition

First validate three separate cases: (i) $\eta_r = \eta_b = 0$ to reproduce the constant-speed baseline, (ii) $\eta_r > 0, \eta_b = 0$ to isolate rejoining acceleration, and (iii) $\eta_r > 0, \eta_b > 0$ with moving obstacles. Only after the controller-level behavior is stable should the point-agent plant be replaced with a multirotor model. This preserves a clean distinction between cognitive/swarm dynamics and vehicle dynamics.